TI202 - Algorithmique et Structures de données 1

TD6 - Fonctions (2h)

Consignes:
- Le travail est **individuel**.
- Le travail est à faire sur papier.
- Prenez le temps de lire et de comprendre les questions avant de commencer à répondre.
- Il est fortement recommandé de se référer aux cours et aux supports de cours pour répondre aux questions.

Acquis d'apprentissage:
À la fin de ce TD, vous serez capable de:
- Déterminer le prototype d'une fonction à partir d'une description fonctionnelle.
- Écrire et appeler des fonctions pour résoudre des problèmes simples.
- Maîtriser la portée des variables ainsi que la notion de variables locales.
- Comprendre à quel moment utiliser les pointeurs en paramètres des fonctions.
- Représenter les variables locales en mémoire.

Nombre aléatoire avec `rand()` et `srand()`

La fonction `rand()` en C est utilisée pour générer des nombres pseudo-aléatoires. Elle fait partie de la bibliothèque standard du langage C et est déclarée dans l'en-tête `<stdlib.h>`. Elle ne prend pas d'argument et retourne un nombre entier aléatoire compris entre 0 et `RAND_MAX` (aussi définie dans `<stdlib.h>` et qui vaut au moins 32767).

`rand()` utilise un algorithme déterministe qui produit toujours la même séquence de nombres si elle est appelée plusieurs fois, sauf si on initialise une graine (*seed*) différente avec la fonction `srand()`.

Afin de s'assurer de toujours avoir une séquence de valeurs différente, il est d'usage d'initialiser le *seed* avec l'heure actuelle du système avec la fonction `time()` (contenue dans le fichier d'en-tête `<time.h>`).

Le programme suivant montre l'utilisation de `rand()` avec une initialisation du *seed* à chaque exécution.

```c
#include <stdio.h>
#include <stdlib.h> /* Pour rand() et srand() */
#include <time.h> /* Pour time() */

int main() {
    srand(time(NULL));  /* Initialise le seed avec l'heure actuelle */
    printf("Nombre aléatoire : %d\n", rand()); /* Nombre aléatoire entre 0 et RAND_MAX */
    printf("Nombre aléatoire : %d\n", rand() % 10); /* Nombre aléatoire entre 0 et 9 */
    return 0;
}
```

Comme `srand()` initialise la séquence pseudo-aléatoire, et elle doit être appelée une seule fois au début du programme. Même avec plusieurs appel à `rand()` dans un même programme.

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main() {
    for (int i = 0; i < 5; i++) {
        srand(time(NULL)); /* À ne pas faire ! */
        printf("%d\n", rand());
    }
    return 0;
}
```

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main() {
    srand(time(NULL)); /* Utilisation correcte */
    for (int i = 0; i < 5; i++) {
        printf("%d\n", rand());
    }
    return 0;
}
```

Échauffement

Exercice 1. On considère le programme suivant, ainsi que les définitions des fonctions `f` et `g`:

```c
#include<stdio.h>
int main () {
    int a = 10;
    int * pa = &a;
    printf("%d\n", a);
    f(a);
    printf("%d\n", a);
    g(pa);
    printf("%d\n", a);
    return 0;
}
```

```c
void f(int a) {
    a++;
}

void g(int * a) {
    (*a)++;
}
```

Représentez toutes les variables en mémoire afin de déterminer les sorties du `main`.

Exercice 2. On considère le prototype des fonctions suivantes:

```c
void f1(int a, int b);
char f2(char a);
float f3(int * arr, int n);
void f4();
```

Les bouts de programmes suivants contiennent des erreurs. Identifiez-les, puis proposez une correction. On suppose que toutes les fonctions sont bien définies.

(a)
```c
/* ... */
int a = 0;
a = f1(3, 4);
printf("%d", a);
/* ... */
```

(b)
```c
/* ... */
char * a;
a = f2('2');
printf("%d", a);
/* ... */
```

(c)
```c
/* ... */
int arr[] = {1, 2, 3};
f3(&arr, 3);
printf("%d", tab[1]);
/* ... */
```

(d)
```c
/* ... */
char c = 'c';
f4(c);
printf("%c", c);
/* ... */
```

Renforcement

Exercice 3. On se propose d'écrire une petite bibliothèque de fonctions pour manipuler des tableaux d'entiers.

(a) Pour chacune des fonctions décrites ci-dessous, proposez son prototype.

- `new_array_int`: Prend en paramètre le nombre d'éléments `n`, renvoie un tableau d'entiers dynamiquement alloué de `n` éléments et initialise les éléments du tableau à 0.
- `free_array_int`: Prend en paramètre un tableau d'entiers `arr` alloué dynamiquement et libère toute la mémoire allouée à ce tableau.
- `print_array_int`: Prend en paramètres un tableau d'entiers `arr` et sa taille `n` et affiche les éléments de `arr` sur une ligne et séparé par des espaces.
- `fill_array_int`: Prend en paramètres un tableau d'entiers `arr` et sa taille `n` et lit chaque élément sur l'entrée standard.
- `random_array_int`: Prend en paramètre le nombre d'éléments `n` et une valeur maximale `max_value`, alloue dynamiquement de la mémoire pour stoker `n` entiers, et rempli le tableau avec des valeurs aléatoire allant de 0 à `max_value`. Cette fonction renvoie l'adresse du premier élément du tableau rempli.

(b) Pour chacune des fonctions dont vous avez donné le prototype, écrivez sa définition.

(c) Compléter le programme suivant afin de tester les fonctions précédemment définies.

```c
#include<stdio.h>
#include <stdlib.h>
#include <time.h>

#define MAX_VALUE 100

/* Déclarations des fonctions utilisateurs (À NE PAS COMPLÉTER) */
/* ... */

int main() {
    int n = 0;
    int * arr1;
    int * arr2;

    /* Initialise le seed de rand() */

    /* Alloue et initialise arr1 à 0 */

```

```
    /* Alloue et initialise arr2 avec des valeurs aléatoire entre 0 et MAX_VALUE */

    /* Affiche les tableaux, arr1 et arr2 */

    /* Libère toute la mémoire allouée sur le tas */

    return 0;
}

/* Définitions des fonctions utilisateurs (À NE PAS COMPLÉTER) */
/* ... */
```

Exercice 4. [Retour du tri sélection] On rappelle que l'algorithme du tri sélection trie un tableau d'entiers, en cherchant à chaque itération le plus petit élément du tableau, l'échange avec l'élément courant du tableau, puis commence l'itération suivante à partir de l'élément suivant.

1. Écrire une fonction `min_pos` qui prend en paramètre un tableau d'entiers `arr` et sa taille `n` et qui renvoie l'indice du plus petit élément dans `arr`.

2. Écrire une fonction `swap_pos` qui prend en paramètre un tableau d'entiers `arr` et deux indices, qu'on suppose *valides*, `i` et `j` du tableau et échange les valeurs à ces indices.

3. En vous aidant des deux fonctions précédentes, écrire une fonction `min_sort` qui prend en paramètre un tableau d'entiers `arr` et sa taille `n` et trie ce tableau selon l'algorithme du tri sélection.

   Indication : *Pour l'itération* `i`*, on recherche le minimum à partir de l'élément* `i` *sur* `n-i` *éléments.*

4. Compléter alors le programme suivant afin de tester les fonctions. Vous devez également utiliser les fonctions sur les tableaux de l'exercice précédent.

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

/* Déclaration des fonctions utilisateurs (À NE PAS COMPLÉTER) */
/* ... */

int main () {
    int n = 0;
    printf("Enter size: ");
    scanf("%d", &n);
    int * arr = NULL;

    /* Initialisation du seed pour rand() */

    /* Allocation et initialisation un tableau arr avec des valeurs aléatoire entre 0 et 100*/

    /* Afficher le tableau initial */

    /* Tri du tableau avec min_sort() */

    /* Afficher le tableau initial */

    /* Libération de la mémoire allouée sur le tas*/

    return 0;
}

```

```
/* Définitions des fonctions utilisateurs (À NE PAS COMPLÉTER) */
/* ... */
```

Approfondissement

Exercice 5. Dans cet exercice, nous écrirons une fonction calculant la liste de tous les nombres premiers plus petits qu'un certain nombre donné en entrée.

Afin d'y parvenir, on utilisera une méthode connue sous le nom de *crible d'Ératosthène*, qui consiste à déterminer les nombres premiers par filtrages successifs.

On peut comprendre cette méthode de la manière suivante: imaginez que vous êtes sur un chemin infini éclairé par une suite de lampadaires numérotés à partir de 2, tous allumés. À côté de chaque lampadaire k, il y a un interrupteur qui a l'effet d'éteindre tous les lampadaires numérotés par un multiple de k, sauf k lui-même. On commence à se promener le long du chemin et, à chaque fois que l'on passe à côté d'un lampadaire allumé, on appuie sur l'interrupteur correspondant. Au fur et à mesure, les seuls lampadaires restant allumés que vous laisserez derrière vous seront exactement ceux correspondants aux nombres premiers.

Dans toute la suite du problème, vous pouvez utiliser les fonctions précédemment définies pour les tableaux.

1. Dans un premier temps, nous allons écrire une fonction `sieve` qui prend en paramètres un tableau d'entiers `arr`, sa taille `n` et un entier `k`. Cette fonction modifie le tableau `arr` en mettant tous les éléments d'indice multiple de `k` à 0, sauf l'élément d'indice `k` lui-même.

   Exemple : *Si* `arr` *est le tableau* $\{0,0,2,3,4,5,6,7,8,9,10\}$, *alors* `sieve(arr, 11, 3)` *modifie* `arr` *en le tableau* $\{0,0,2,3,4,5,0,7,8,0,10\}$

2. Nous voulons ensuite écrire une fonction `primes` qui prend en paramètre un entier `n` et qui renvoie une liste de tous les nombres premiers plus petits que `n` dans un tableau.

   Afin d'y parvenir, écrivez les fonctions auxiliaires suivantes:

   (a) Une fonction `init_array` qui prend en paramètre un entier `n` et renvoie le tableau $\{0,0,2,3,\ldots,n-1\}$, que vous allouerez dans le tas. Les premier et deuxième de ce tableau sont initialisés à 0.

   Exemple : `init_array(10)` *renvoie le tableau* $\{0,0,2,3,4,5,6,7,8,9\}$.

   (b) Une fonction `count_zeroes` qui prend en paramètres un tableau `arr` de `n` entiers et renvoie le nombre de 0 présents dans `arr`.

   Exemple : *Si* `arr` *est le tableau* $\{0,3,-4,0,0,7\}$, *alors* `count_zeroes(arr, 6)` *renvoie* 3.

   (c) Une fonction `compress` qui prend en paramètres un tableau `arr` de `n` entiers, et qui renvoie un tableau obtenu à partir de `arr` en supprimant tous les 0.

   Indication : *Trouvez la bonne taille à allouer pour le nouveau tableau en vous aidant de la fonction* `count_zeroes`.

   Exemple : *Si* `arr` *est le tableau* $\{0,3,-4,0,0,7\}$, *alors* `compress(arr, 6)` *renvoie le tableau* $\{3,-4,7\}$.

   (d) Une fonction `erato_sieve` qui prend en paramètre un entier `n` et qui renvoie le tableau $\{0,0,2,3,\ldots,n-1\}$ auquel on applique le *crible d'Ératosthène*. Attention, à l'indice 1 de ce tableau on a un 0 car par définition, 1 n'est pas premier!

   Exemple : `erato_sieve(10)` *renvoie le tableau* $\{0,0,2,3,0,5,0,7,0,0\}$.

Écrire alors la fonction `primes` qui prend en paramètre un entier `n` et qui renvoie une liste de tous les nombres premiers plus petits que `n` dans un tableau.

Exemple : `primes(100)` *renvoie le tableau* $\{2, 3, 5, 7\}$.

3. Modifier la fonction `compress` et la fonction `primes` afin de pouvoir garder en mémoire la taille du tableau renvoyé par la fonction `primes`.

4. Écrire un programme principal, qui lit sur l'entrée standard un entier `n` et qui affiche la liste de tous les nombres premiers plus petits que `n` ainsi que leur nombre.

   Par exemple, si `n = 10`, le programme produira en sortie:

```
2 3 5 7 11 13 17 19 23 29 31 37 41 43 47 53 59 61 67 71 73 79 83 89 97
25
```